Universidad Tecnológica Metropolitana

Desarrollo de Software Multiplataforma · Grupo 7A

Bases de Datos en la Nube · Unidad II

UTM Desk NoSQL Cloud: gestión de incidencias del campus con MongoDB Atlas

Arroyo León Luis Diego

Can Domínguez Víctor Emmanuel

Montalvo Canché Emir Emanuel

Vargas Guzmán Kevin Aarón

Docente: Lizandro Israel Reyes Carrillo

16 de octubre de 2026

UTM Desk NoSQL Cloud: gestión de incidencias del campus con MongoDB Atlas

Problemática

El campus recibe incidencias de tres fuentes con datos muy distintos: cómputo y redes, infraestructura física y sistemas académicos. Las hojas de cálculo y los correos dispersan la trazabilidad, no conservan la bitácora de cada intervención y no permiten medir el tiempo de resolución ni detectar fallas que se repiten en un mismo espacio. UTM Desk centraliza esas incidencias en una base documental en la nube, guarda las evidencias fuera de la base y ofrece búsqueda textual y analítica por edificio.

El objetivo medible fue demostrar en vivo el ciclo completo de una incidencia: alta, evidencia, seguimiento, etiquetas, búsqueda, analítica y depuración automática, sobre infraestructura desplegada en la nube y no sobre una laptop.

Justificación

Las tres categorías comparten campos universales (título, descripción, prioridad, ubicación, bitácora), pero cada una necesita atributos propios. En un modelo relacional esto exige una tabla por categoría o columnas casi siempre vacías, y consultar un expediente completo requiere varios *joins*. En un modelo documental, cada incidencia es un solo documento con un campo `detalle_especifico` polimórfico y con la bitácora, las etiquetas y las evidencias embebidas, de modo que el expediente se lee en una sola operación (MongoDB, Inc., s. f.-a).

Los archivos no se guardan en la base: MongoDB conserva solo los metadatos y la ruta del objeto en un depósito privado de Supabase Storage (Supabase, s. f.), lo que mantiene la base muy por debajo del límite de 512 MB del nivel gratuito de Atlas.

Requisitos

Requisitos funcionales

Tabla 1

Requisitos funcionales y su ruta de la API

Requisito	Descripción	Ruta
RF-01/02	Registrar incidencia con validación en dos capas	`POST /api/incidencias`
RF-03	Consecutivo diario atómico en la colección `contadores`	(interno al alta)
RF-04	Cargar evidencia (JPEG, PNG o PDF, máximo 4 MB)	`POST /api/incidencias/:codigo/evidencias`
RF-05/06	Consultar expediente y listar con filtros y paginación	`GET /api/incidencias[/:codigo]`
RF-07	Cambiar estado, horas y bitácora sin condición de carrera	`PATCH /api/incidencias/:codigo/estado`
RF-08	Etiquetas sin duplicados	`POST|DELETE …/tags`
RF-09	Búsqueda textual en español ordenada por relevancia	`GET /api/incidencias/buscar?q=`
RF-10	Analítica por edificio y etiquetas más frecuentes	`GET /api/analitica/edificios|tags`
RF-11	Depuración automática a los 60 días de la resolución	índice TTL
RF-12	Editar campos universales y borrar (coordinador)	`PATCH|DELETE /api/incidencias/:codigo`
RF-13	Sesión con JWT y permisos por rol	`POST /api/auth/login`
RF-14/15	Datos de demostración, exportables e interfaz web	scripts y web

Requisitos no funcionales

Rendimiento (listados y búsqueda con índice), almacenamiento dentro de 512 MB, seguridad (contraseñas con bcrypt, JWT, *helmet*, CORS restringido, límite de peticiones, validación de entrada y depósito privado), integridad (validación en dos capas y cambios de estado atómicos), disponibilidad en nube, usabilidad en español y móvil, mantenibilidad (arquitectura limpia y pruebas automáticas) y entrega en formato APA 7.ª edición.

Roles

Tabla 2

Roles y permisos

Rol	Puede
Reportante	Crear incidencias, ver las suyas y subir evidencia a las suyas
Técnico	Ver todas, cambiar estado, horas y bitácora, etiquetas y evidencias
Coordinador	Todo lo del técnico, cerrar, editar campos universales, borrar y consultar analítica
Directivo	Solo lectura, búsqueda y analítica

Tecnologías y arquitectura

Tabla 3

Tecnologías elegidas y razón

Capa	Tecnología	Razón
Base de datos	MongoDB Atlas (M0)	Modelo documental polimórfico, `$jsonSchema`, índices de texto en español y TTL (MongoDB, Inc., s. f.-b, s. f.-c)
Archivos	Supabase Storage (privado)	Binarios fuera de la base y URL firmadas temporales
API	Node.js, Express 5, controlador nativo de MongoDB	Contrato REST simple y control directo de las operaciones atómicas
Validación	Zod	Esquemas por categoría y rechazo de campos extra y operadores NoSQL (Colin McDonnell, s. f.)
Interfaz	React, Vite, Tailwind CSS, Recharts	Formulario dinámico, diseño responsivo y gráficas (Meta Platforms, s. f.)
Pruebas	Vitest, Supertest, Newman	Pruebas de integración contra Atlas real y banco de pruebas Postman
Nube	Vercel (API y web)	La demostración no depende de la laptop (Vercel, s. f.)

El servidor sigue una arquitectura limpia con cuatro capas: `domain` (enumeraciones, transiciones de estado y esquemas), `application` (casos de uso), `infrastructure` (repositorios de Mongo, almacén de Supabase, seguridad) e `interface` (rutas y *middlewares* de Express). Una raíz de composición conecta los puertos con sus adaptadores, lo que permite probar los casos de uso con dobles y desplegar la misma aplicación como función sin servidor.

Decisión acordada con el docente: el consecutivo diario se mantiene en MongoDB (colección `contadores`, operación `$inc` con `upsert`) y no en Redis, con lo que se evita una dependencia y una cuenta adicionales.

Modelo de datos

La colección `incidencias_campus` tiene un validador `$jsonSchema` estricto (`validationLevel: strict`, `validationAction: error`) con una rama `oneOf` por categoría para `detalle_especifico`; `fecha_resolucion` solo existe en incidencias resueltas o cerradas. Zod aplica la misma regla antes de llegar a la base, de modo que Mongo actúa como segunda línea de defensa.

Tabla 4

Índices de la colección incidencias_campus

Índice	Clave	Uso
`idx_cat_estado_edificio`	categoría, estado, edificio	Listados filtrados
`idx_ttl_resolucion`	fecha_resolucion (TTL de 60 días)	Depuración automática
`idx_texto_es`	título (peso 10), descripción (peso 5), idioma español	Búsqueda por relevancia
`idx_codigo`	código (único)	Consulta de expediente
`idx_reportante`	reportante, fecha de creación descendente	Mis incidencias

Las tres categorías, con ejemplos reales del conjunto de datos, se muestran a continuación (bitácora abreviada a su primera entrada y una evidencia).

Ejemplo de cómputo y redes


```
{
  "_id": {
    "$oid": "6ad1c6a46e02164f43308dcd"
  },
  "codigo": "INC-2026-0003",
  "consecutivo_diario": 1,
  "titulo": "Switch del laboratorio LC-2 sin respuesta",
  "descripcion": "Desde las 8:00 los equipos del laboratorio no obtienen red ni acceso a internet.",
  "categoria": "computo_redes",
  "prioridad": "alta",
  "estado": "resuelta",
  "reportante": {
    "usuario_id": {
      "$oid": "6ad1c69c6e02164f43308dc4"
    },
    "nombre": "Mtra. Ana Pech",
    "tipo": "docente",
    "correo": "ana.pech@utmetropolitana.edu.mx"
  },
  "ubicacion": {
    "edificio": "Laboratorios de Cómputo",
    "piso": 1,
    "espacio": "LC-2"
  },
  "detalle_especifico": {
    "tipo_falla": "switch",
    "direccion_ip": "10.20.2.1",
    "vlan": 20,
    "puerto_uplink": "Gi1/0/24",
    "equipos_afectados": 28
  },
  "tags": [
    "switch",
    "sin-red",
    "laboratorio",
    "recurrente"
  ],
  "historial_seguimiento": [
    {
      "fecha": {
        "$date": "2026-09-15T05:39:21.675Z"
      },
      "autor": {
        "usuario_id": {
          "$oid": "6ad1c69c6e02164f43308dc4"
```

```
        },
        "nombre": "Mtra. Ana Pech"
      },
      "estado_nuevo": "abierta",
      "comentario": "Incidencia registrada",
      "horas": 0
    }
  ],
  "evidencias_adjuntas": [
    {
      "nombre_archivo": "switch-lc2.png",
      "tipo_mime": "image/png",
      "peso_kb": 1.02,
      "storage_path": "INC-2026-0003/3cd1a900-1691-4933-8e3e-04859f75ad3a-switch-lc2.png",
      "url_storage": "https://…/firmada",
      "url_expira": {
        "$date": "2026-10-16T07:39:25.806Z"
      },
      "fecha_carga": {
        "$date": "2026-09-15T06:09:21.675Z"
      }
    }
  ],
  "horas_invertidas": 3.5,
  "fecha_creacion": {
    "$date": "2026-09-15T05:39:21.675Z"
  },
  "fecha_actualizacion": {
    "$date": "2026-09-15T10:39:21.675Z"
  },
  "asignado_a": {
    "usuario_id": {
      "$oid": "6ad1c69c6e02164f43308dc7"
    },
    "nombre": "Luis Cauich"
  },
  "fecha_resolucion": {
    "$date": "2026-09-15T10:39:21.675Z"
  }
}
```


Ejemplo de infraestructura


```
{
  "_id": {
    "$oid": "6ad1c6a46e02164f43308dcb"
  },
  "codigo": "INC-2026-0001",
  "consecutivo_diario": 1,
  "titulo": "Filtración en el techo del aula A-203",
  "descripcion": "Gotea agua sobre las bancas del fondo cada vez que llueve; el piso queda resbaloso.",
  "categoria": "infraestructura",
  "prioridad": "media",
  "estado": "resuelta",
  "reportante": {
    "usuario_id": {
      "$oid": "6ad1c69c6e02164f43308dc4"
    },
    "nombre": "Mtra. Ana Pech",
    "tipo": "docente",
    "correo": "ana.pech@utmetropolitana.edu.mx"
  },
  "ubicacion": {
    "edificio": "Edificio A",
    "piso": 2,
    "espacio": "A-203"
  },
  "detalle_especifico": {
```

```
    "tipo_dano": "filtracion",
    "riesgo_seguridad": "medio",
    "requiere_contratista": true,
    "contratista_externo": "Impermeabilizaciones del Sureste",
    "insumos_requeridos": [
      "impermeabilizante acrílico",
      "sellador de juntas"
    ]
  },
  "tags": [
    "filtracion",
    "techo",
    "recurrente"
  ],
  "historial_seguimiento": [
    {
      "fecha": {
        "$date": "2026-09-14T06:39:21.675Z"
      },
      "autor": {
        "usuario_id": {
          "$oid": "6ad1c69c6e02164f43308dc4"
        },
        "nombre": "Mtra. Ana Pech"
      },
      "estado_nuevo": "abierta",
      "comentario": "Incidencia registrada",
      "horas": 0
    }
  ],
  "evidencias_adjuntas": [
    {
      "nombre_archivo": "foto-techo-a203.png",
      "tipo_mime": "image/png",
      "peso_kb": 1.02,
      "storage_path": "INC-2026-0001/bbf360f9-4cd2-4d51-9815-6990855fb73c-foto-techo-a203.png",
      "url_storage": "https://…/firmada",
      "url_expira": {
        "$date": "2026-10-16T07:39:24.746Z"
      },
      "fecha_carga": {
        "$date": "2026-09-14T07:09:21.675Z"
      }
    }
  ],
  "horas_invertidas": 5,
  "fecha_creacion": {
    "$date": "2026-09-14T06:39:21.675Z"
  },
  "fecha_actualizacion": {
    "$date": "2026-09-20T06:39:21.675Z"
  },
  "asignado_a": {
    "usuario_id": {
      "$oid": "6ad1c69c6e02164f43308dc7"
    },
    "nombre": "Luis Cauich"
  },
  "fecha_resolucion": {
    "$date": "2026-09-20T06:39:21.675Z"
  }
}
```


Ejemplo de sistemas académicos


```
{
  "_id": {
    "$oid": "6ad1c6a46e02164f43308dcc"
```

```
  },
  "codigo": "INC-2026-0002",
  "consecutivo_diario": 1,
  "titulo": "Cuentas de correo institucional bloqueadas (periodo 2026-2)",
  "descripcion": "Decenas de estudiantes no pueden iniciar sesión en su correo institucional desde el
lunes.",
  "categoria": "academico",
  "prioridad": "alta",
  "estado": "resuelta",
  "reportante": {
    "usuario_id": {
      "$oid": "6ad1c69c6e02164f43308dc6"
    },
    "nombre": "Pedro Canul",
    "tipo": "administrativo",
    "correo": "pedro.canul@utmetropolitana.edu.mx"
  },
  "ubicacion": {
    "edificio": "Administrativo",
    "piso": 1,
    "espacio": "Coordinación TIC"
  },
  "detalle_especifico": {
    "plataforma": "correo_institucional",
    "periodo_escolar": "2026-2",
    "estudiantes_impactados": 85
  },
  "tags": [
    "correo",
    "acceso",
    "recurrente"
  ],
  "historial_seguimiento": [
    {
      "fecha": {
        "$date": "2026-09-15T04:39:21.675Z"
      },
      "autor": {
        "usuario_id": {
          "$oid": "6ad1c69c6e02164f43308dc6"
        },
        "nombre": "Pedro Canul"
      },
      "estado_nuevo": "abierta",
      "comentario": "Incidencia registrada",
      "horas": 0
    }
  ],
  "evidencias_adjuntas": [
    {
      "nombre_archivo": "bloqueos-correo.png",
      "tipo_mime": "image/png",
      "peso_kb": 1.02,
      "storage_path": "INC-2026-0002/49f3f798-c93a-4c2c-9617-15360692a5a8-bloqueos-correo.png",
      "url_storage": "https://…/firmada",
      "url_expira": {
        "$date": "2026-10-16T07:39:25.422Z"
      },
      "fecha_carga": {
        "$date": "2026-09-15T05:09:21.675Z"
      }
    }
  ],
  "horas_invertidas": 3,
  "fecha_creacion": {
    "$date": "2026-09-15T04:39:21.675Z"
  },
  "fecha_actualizacion": {
    "$date": "2026-09-16T00:39:21.675Z"
  },
  "asignado_a": {
```

```
    "usuario_id": {
      "$oid": "6ad1c69c6e02164f43308dc8"
    },
    "nombre": "Marisol Be"
  },
  "fecha_resolucion": {
    "$date": "2026-09-16T00:39:21.675Z"
  }
}
```


La base de demostración contiene 24 incidencias sintéticas distribuidas en cinco edificios o zonas, las cuatro prioridades y los cinco estados, 12 con fecha de resolución, 18 con evidencias y 26 etiquetas distintas. El script `db:verify` comprueba estas condiciones de forma automática.

Implementación de los casos de uso

Tabla 5

Operaciones de MongoDB por caso de uso

Caso de uso	Operación
CU-01 Alta	`findOneAndUpdate` con `$inc` y `upsert` en `contadores`, luego `insertOne` validado por Zod y por `$jsonSchema`
CU-02 Evidencia	Tipo real por firma de bytes, subida al depósito, un `$push` en `evidencias_adjuntas`; URL firmada regenerada en cada lectura
CU-03 Estado	Un solo `updateOne` con filtro `{ codigo, estado: estadoLeido }`, `$set`, `$push` a la bitácora e `$inc` de horas; 409 si otra persona cambió el estado
CU-04 Etiquetas	`$addToSet` (normalizada a minúsculas) y `$pull`
CU-05 Búsqueda	`$text` con `score` proyectado con `$meta` y orden por relevancia
CU-06 Analítica	Pipeline 1: `$match`, `$addFields` con `$dateDiff`, `$group`, `$sort`. Pipeline 2: `$unwind`, `$group`, `$sort`, `$limit` 10
CU-07 Depuración	Índice TTL sobre `fecha_resolucion` con `expireAfterSeconds: 5184000`

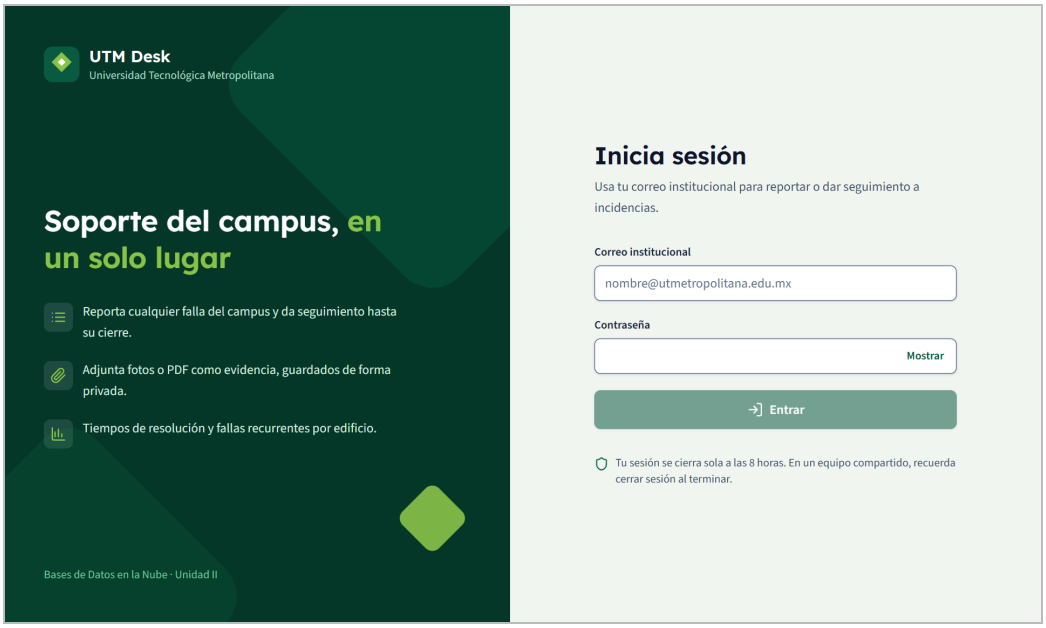
La prueba de concurrencia lanza dos cambios de estado simultáneos contra Atlas: uno recibe 200 y el otro 409, y las horas y la bitácora quedan correctas. La demostración del TTL siembra un documento resuelto hace 61 días y otro hace 59; el primero desaparece en unos diez segundos y el segundo permanece.

Interfaz de usuario

La interfaz está en español, usa el verde institucional con contraste verificado, objetivos táctiles de 44 px y foco visible, y funciona en móvil sin desplazamiento horizontal de la página. A continuación se presentan las pantallas principales.

Figura 1

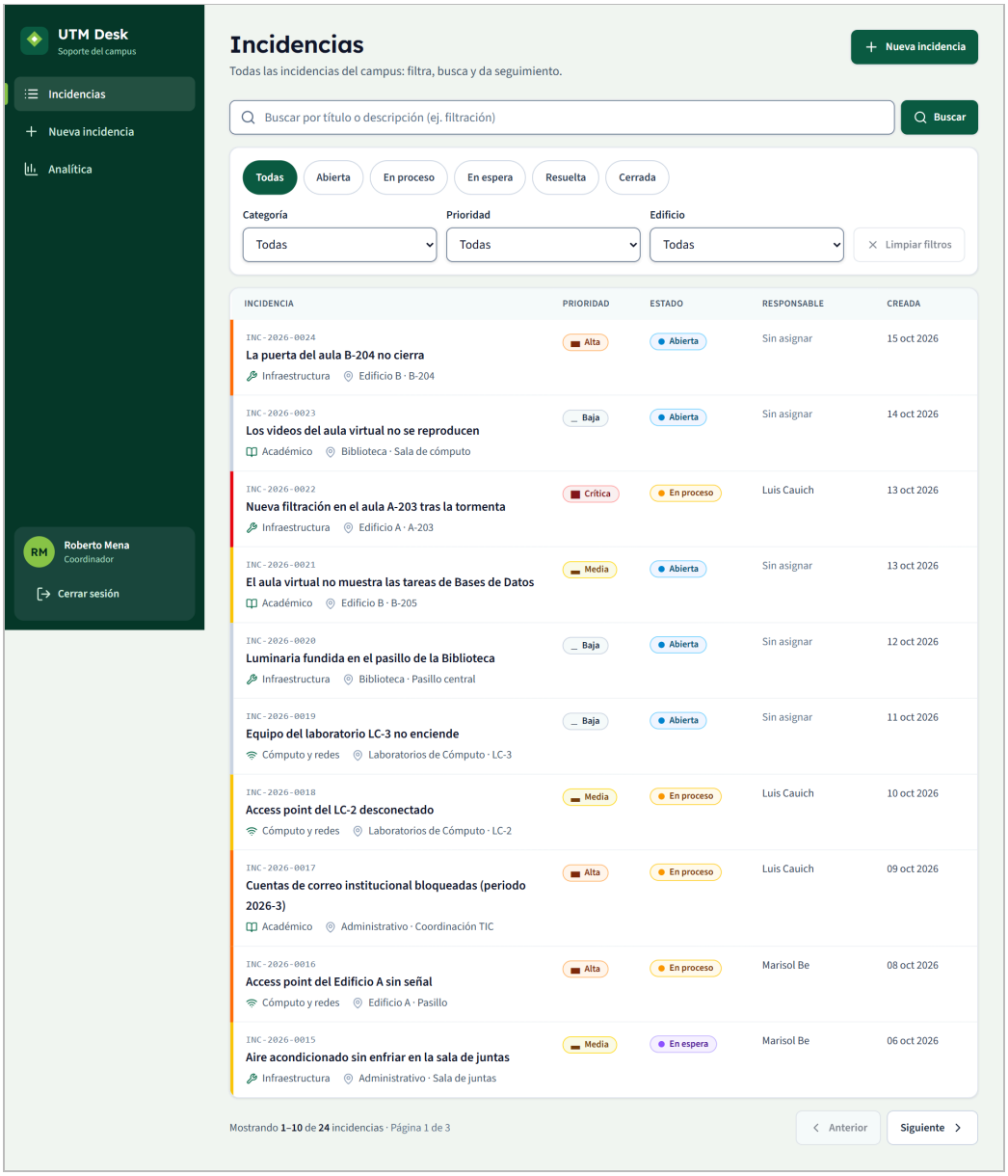
Pantalla de inicio de sesión



Nota. Captura propia del sistema local conectado a MongoDB Atlas y Supabase, con datos ficticios.

Figura 2

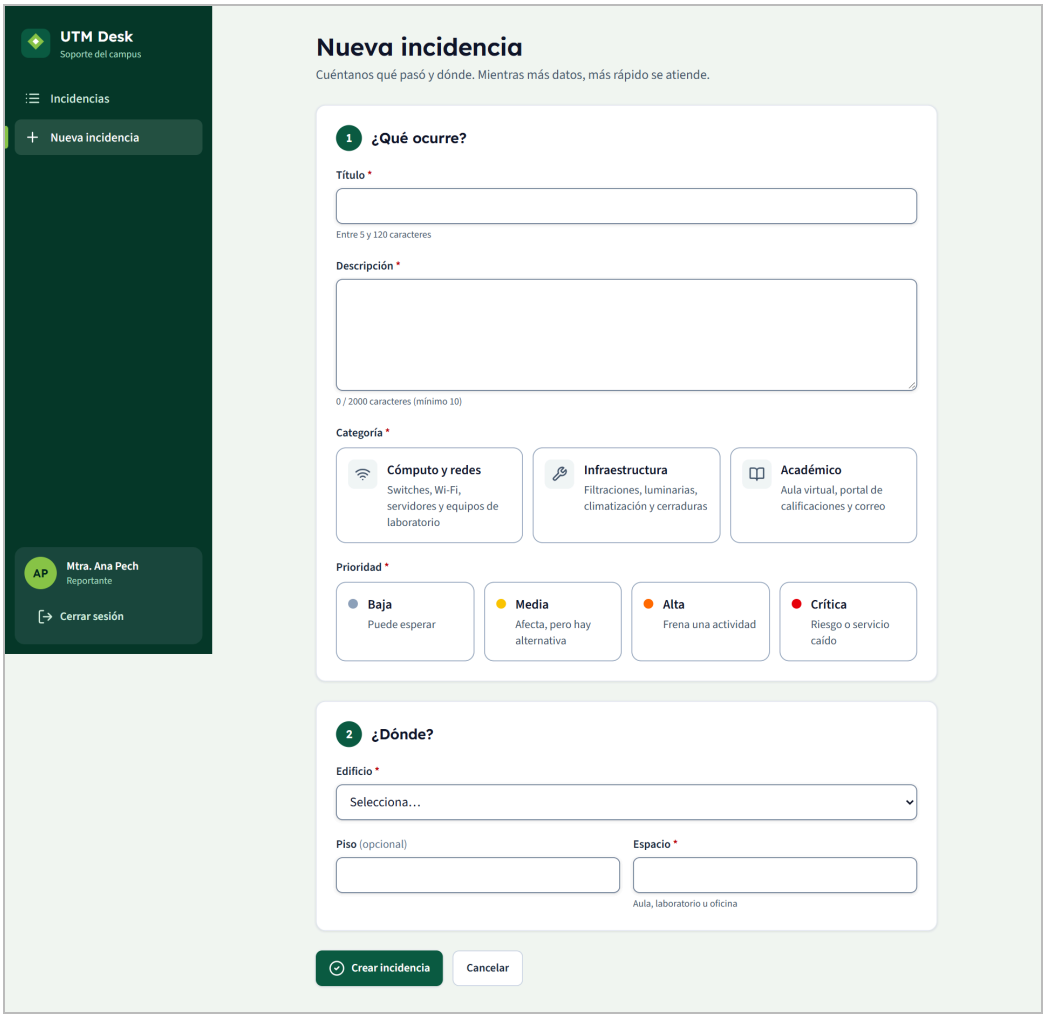
Lista de incidencias con filtros y paginación (rol coordinador)



Nota. Captura propia del sistema local conectado a MongoDB Atlas y Supabase, con datos ficticios.

Figura 3

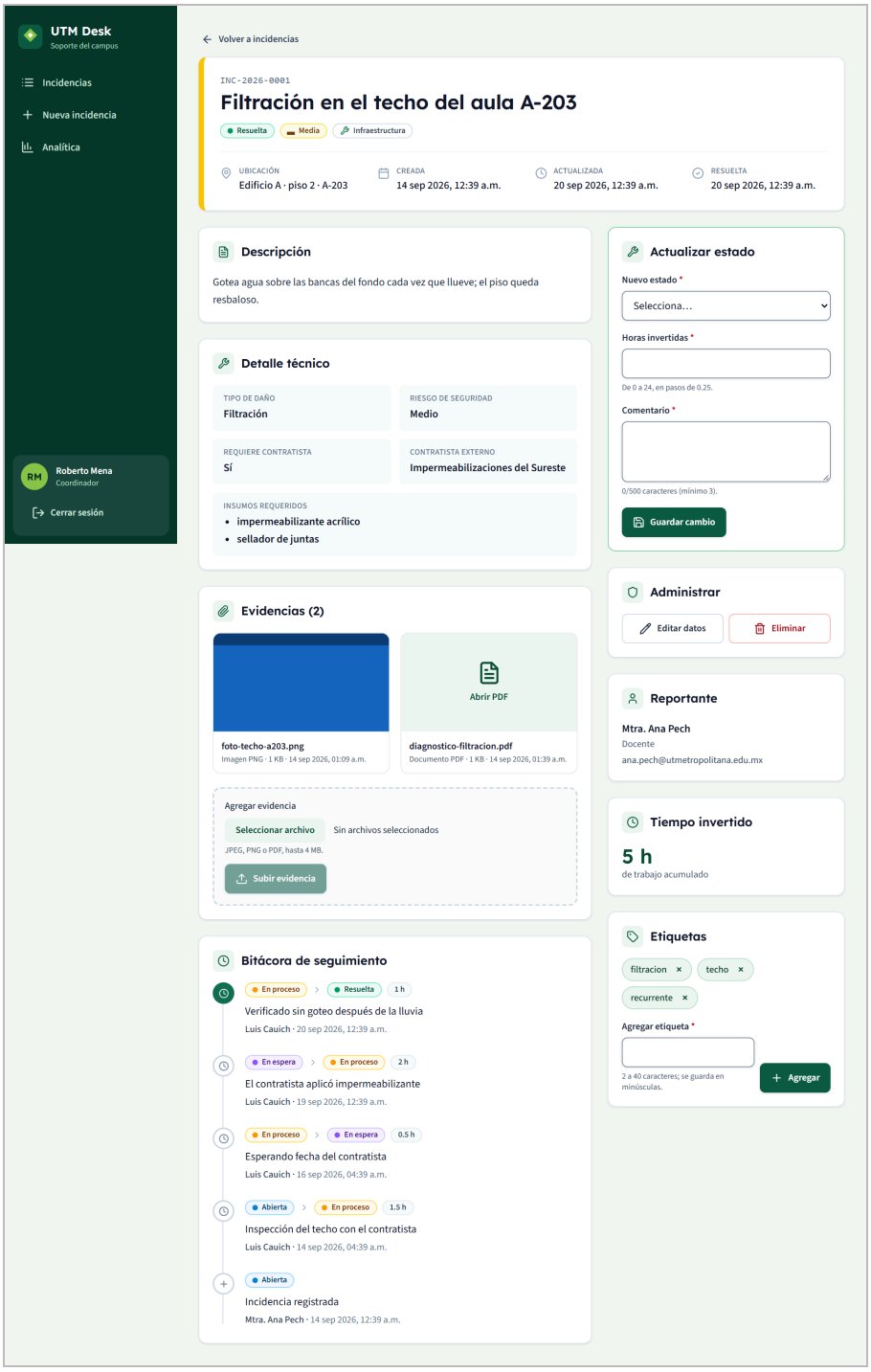
Formulario de nueva incidencia con bloque dinámico por categoría (rol reportante)



Nota. Captura propia del sistema local conectado a MongoDB Atlas y Supabase, con datos ficticios.

Figura 4

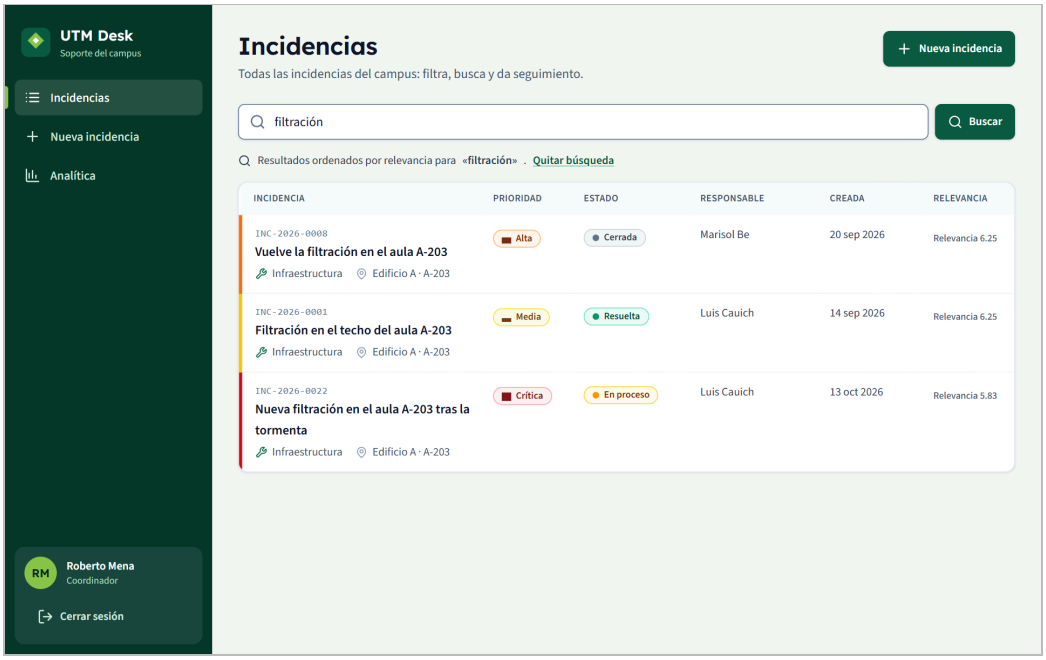
Expediente con bitácora, evidencias, etiquetas y acciones de personal (rol coordinador)



Nota. Captura propia del sistema local conectado a MongoDB Atlas y Supabase, con datos ficticios.

Figura 5

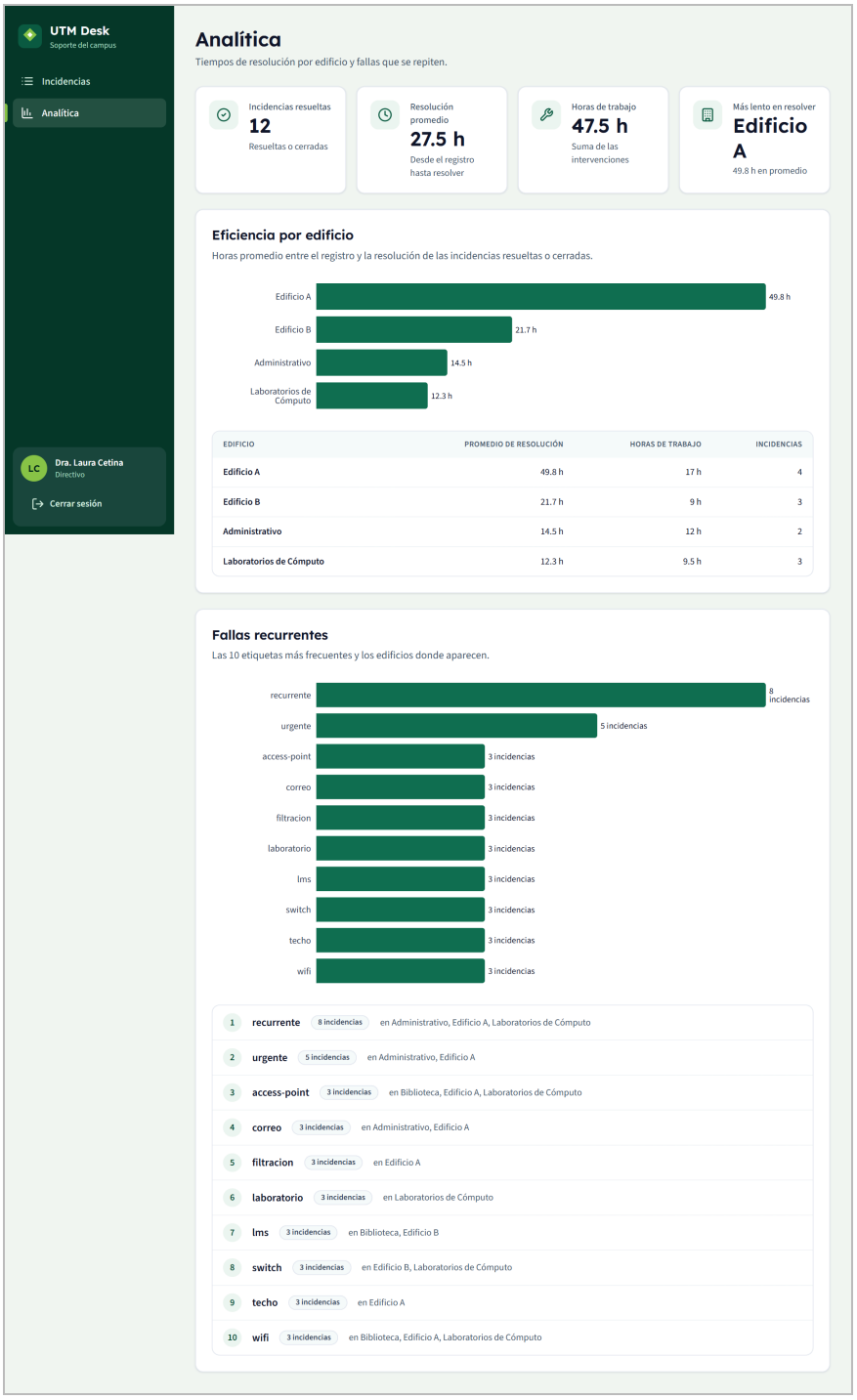
Resultados de búsqueda ordenados por relevancia



Nota. Captura propia del sistema local conectado a MongoDB Atlas y Supabase, con datos ficticios.

Figura 6

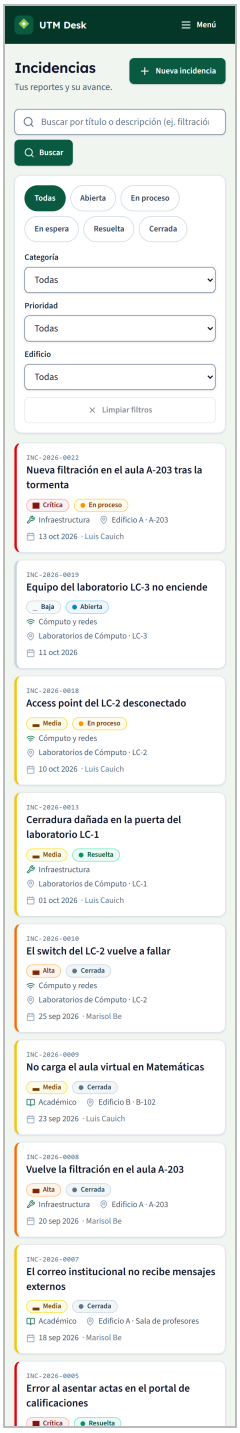
Analítica de eficiencia por edificio y fallas recurrentes (rol directivo)



Nota. Captura propia del sistema local conectado a MongoDB Atlas y Supabase, con datos ficticios.

Figura 7

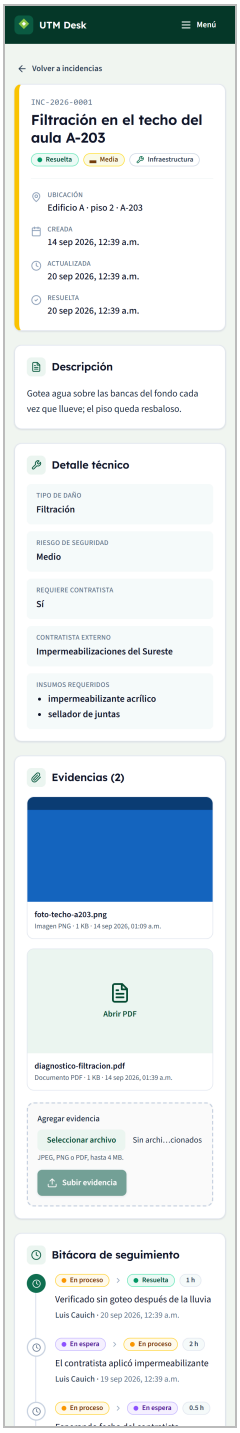
Lista de incidencias en un teléfono



Nota. Captura propia del sistema local conectado a MongoDB Atlas y Supabase, con datos ficticios.

Figura 8

Expediente en un teléfono



Nota. Captura propia del sistema local conectado a MongoDB Atlas y Supabase, con datos ficticios.

Banco de pruebas

La colección de Postman se ejecuta con Newman y cubre 63 peticiones agrupadas por caso de uso, con 125 aserciones. La última corrida contra la API local terminó con 125 de 125 aserciones correctas y 0 fallos; la misma colección también se ejecutó contra la API desplegada en Vercel con el mismo resultado. Además, 351 pruebas de Vitest se ejecutan contra Atlas y Supabase reales, usando colecciones temporales.

Tabla 6

Códigos de estado cubiertos por la colección

Código	Situación probada
200 / 201	Lectura, actualización, borrado y alta correctos
400	Campo faltante, enumeración inválida, `detalle_especifico` de otra categoría, horas fuera de rango, archivo no permitido o vacío
401	Sin token, token inválido o credenciales incorrectas
403	Rol sin permiso o reportante que consulta una incidencia ajena
404	Incidencia inexistente o ya depurada
409	Cambio de estado concurrente (probado de forma determinista en las pruebas de integración)

Despliegue y seguridad

La API y la web están desplegadas en Vercel como dos proyectos independientes que se conectan a MongoDB Atlas y a Supabase mediante variables de entorno marcadas como sensibles. Atlas usa un usuario de base de datos con permiso `readWrite` únicamente sobre la base `utmdesk`. La revisión de seguridad documentó 20 controles verificados, entre ellos contraseñas con bcrypt, JWT de 8 horas, roles validados en el servidor, límite de intentos de inicio de sesión, rechazo de operadores NoSQL por validación estricta, *helmet*, CORS restringido, depósito privado y secretos fuera del repositorio (OWASP Foundation, s. f.). Riesgos aceptados: el límite de peticiones es por instancia en el entorno sin servidor y la sesión se guarda en `localStorage`.

Verificación en Atlas

Con una consulta `explain("executionStats")` se comprobó que los listados, la búsqueda, el código y las incidencias del reportante usan índice:

Tabla 7

Planes de ejecución observados en Atlas

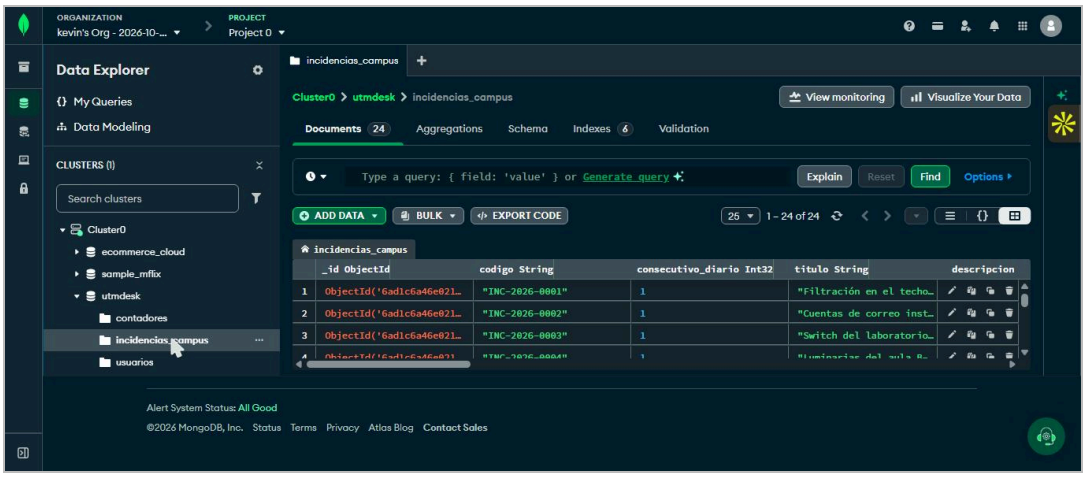
Consulta	Plan y documentos examinados
Listado filtrado (categoría + estado + edificio)	FETCH → IXSCAN(idx_cat_estado_edificio) · examinados 0 docs / 0 claves · devueltos 0
Búsqueda de texto «filtración»	TEXT_MATCH(idx_texto_es) → FETCH → IXSCAN(idx_texto_es) · examinados 3 docs / 3 claves · devueltos 3
Mis incidencias (reportante)	FETCH → IXSCAN(idx_reportante) · examinados 11 docs / 11 claves · devueltos 11

Capturas de MongoDB Atlas

Las siguientes capturas muestran la colección en Atlas (Data Explorer, clúster Cluster0, base `utmdesk`): sus 24 documentos, los seis índices (el predeterminado más los cinco del proyecto, con el TTL y el de texto) y el validador activo con acción de error y nivel estricto.

Figura 9

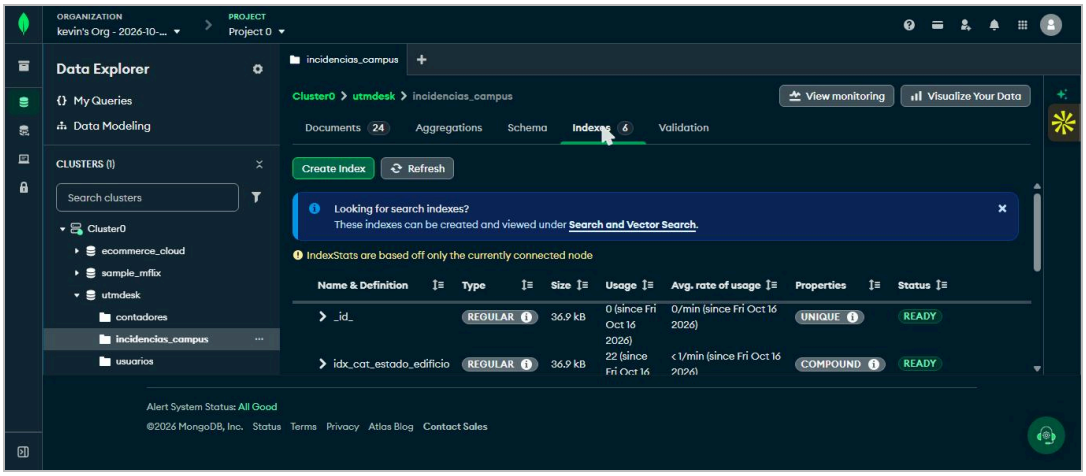
Colección incidencias_campus con sus 24 documentos



Nota. Captura de MongoDB Atlas, 19 de octubre de 2026.

Figura 10

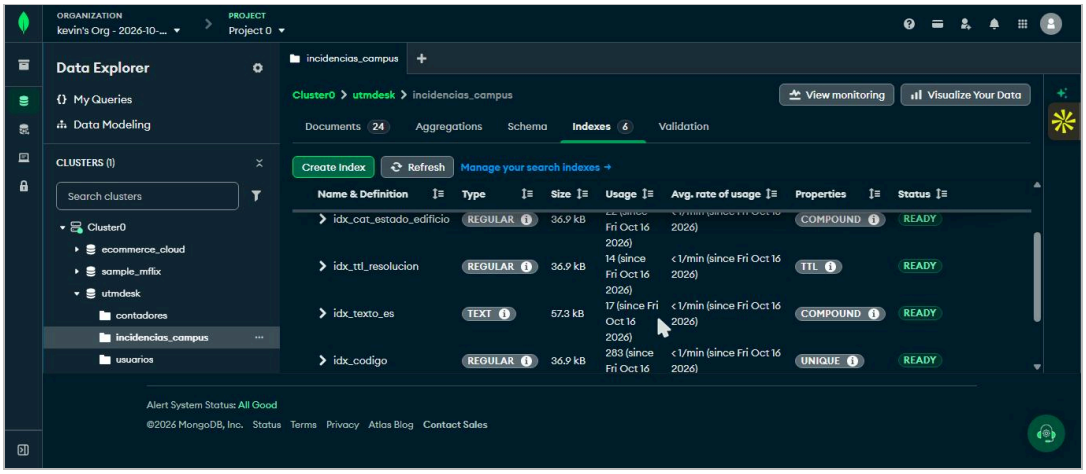
Índices de la colección: predeterminado y compuesto por categoría, estado y edificio



Nota. Captura de MongoDB Atlas, 19 de octubre de 2026.

Figura 11

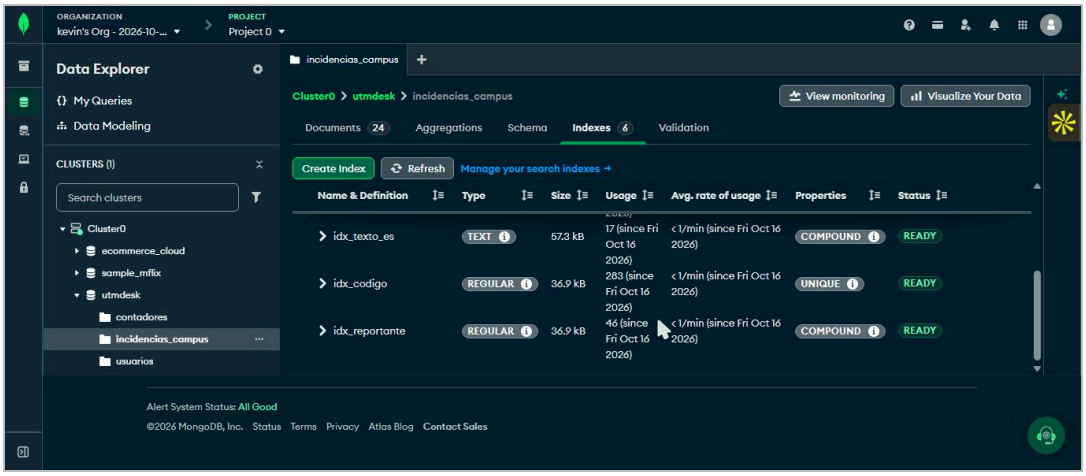
Índices compuesto, TTL de resolución, de texto y de código



Nota. Captura de MongoDB Atlas, 19 de octubre de 2026.

Figura 12

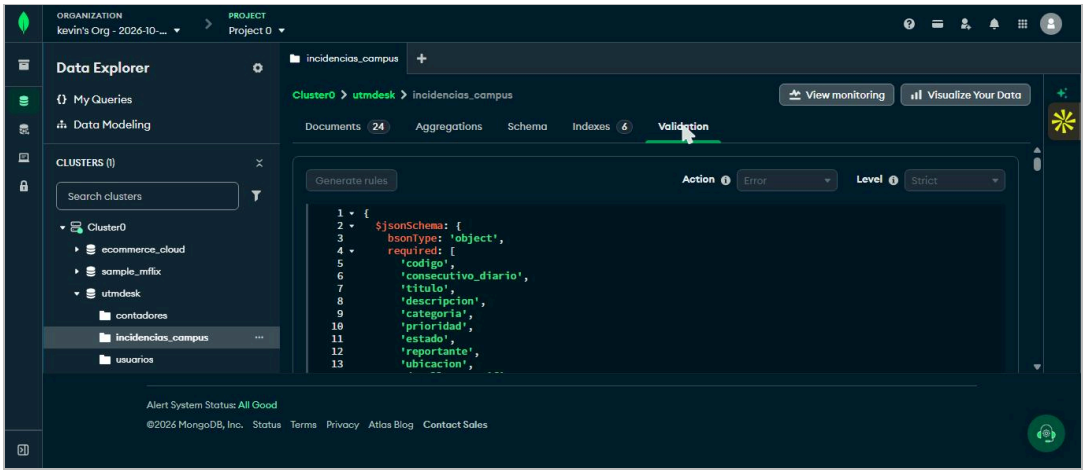
Índices de texto, de código único y por reportante



Nota. Captura de MongoDB Atlas, 19 de octubre de 2026.

Figura 13

Validador $jsonSchema con acción Error y nivel Strict



Nota. Captura de MongoDB Atlas, 19 de octubre de 2026.

Distribución del trabajo

El proyecto se repartió por caso de uso del enunciado para que cada integrante defienda una parte completa: desde la interfaz hasta la operación de MongoDB que la respalda. Todos conocen a grandes rasgos las partes de los demás.

Tabla 8

Parte del proyecto que defiende cada integrante

Integrante	Casos de uso y parte	Archivos principales
Vargas Guzmán Kevin Aarón	CU-01 Alta de incidencias y contador; arquitectura, seguridad y despliegue	`registrarIncidencia.js`, `contadorMongo.js`, `aplicacion.js`, `api/index.js`
Arroyo León Luis Diego	CU-03 Estado, bitácora y concurrencia; roles y banco de pruebas	`actualizarEstado.js`, `transiciones.js`, `incidenciasRepo.js`, colección Postman
Can Domínguez Víctor Emmanuel	CU-02 Evidencias en Supabase y CU-04 Etiquetas	`cargarEvidencia.js`, `detectarTipo.js`, `almacenSupabase.js`, `etiquetasRepo.js`
Montalvo Canché Emir Emanuel	CU-05 Búsqueda, CU-06 Analítica y CU-07 Depuración; índices y datos de ejemplo	`buscarIncidencias.js`, `analiticaRepo.js`, `indices.js`, `demo-ttl.js`

La guía de defensa del repositorio (`docs/guia-defensa.md`) reúne para cada integrante un guion breve, las preguntas probables con su respuesta, los cambios que podrían pedirle en vivo con el archivo donde se hacen y la prueba que los comprueba.

Conclusiones individuales

Borrador de ejemplo: cada integrante debe reescribir su conclusión con sus propias palabras antes de entregar.

Vargas Guzmán Kevin Aarón

Mi parte fue el alta de incidencias y la arquitectura. Aprendí que un modelo documental con un campo polimórfico exige validar en dos capas: Zod ofrece errores claros al usuario y el `$jsonSchema` protege la colección aunque alguien la modifique sin pasar por la API. También entendí por qué el contador atómico con `$inc` y `upsert` evita números repetidos sin necesidad de transacciones. La mayor dificultad fue desplegar la API en Vercel: Atlas rechazaba las conexiones hasta que ajustamos el acceso de red, y aprendimos a manejar variables de entorno

sensibles y límites de tamaño de petición. Considero que separar dominio, aplicación e infraestructura nos permitió probar cada regla sin depender de la base.

Arroyo León Luis Diego

Mi parte fue el cambio de estado y la bitácora. Aprendí que la condición de carrera se resuelve incluyendo el estado leído en el filtro de un único `updateOne`, de modo que si otra persona lo cambió no se escribe nada y se responde 409. Entendí que la bitácora, las horas y la fecha de resolución deben actualizarse en la misma operación para que el documento nunca quede a medias. La dificultad principal fue demostrar el 409 de forma confiable: en la interfaz depende del azar, así que lo probamos con dos cambios simultáneos contra Atlas. También armé la colección de Postman, que nos dio una forma repetible de comprobar códigos 200, 400, 401, 403, 404 y 409.

Can Domínguez Víctor Emmanuel

Mi parte fue la carga de evidencias y las etiquetas. Aprendí que no se debe confiar en la extensión ni en el tipo que declara el navegador: leemos los primeros bytes del archivo para conocer su tipo real y así rechazamos un ejecutable renombrado como imagen. Entendí por qué los binarios viven en un depósito privado de Supabase y en Mongo solo se guardan los metadatos, y cómo las URL firmadas caducan y se regeneran en cada consulta. En las etiquetas aprendí que `$addToSet` evita duplicados sin lógica adicional. La dificultad fue evitar archivos huérfanos cuando falla alguno de los dos pasos, por lo que borramos el objeto si Mongo no registra la evidencia.

Montalvo Canché Emir Emanuel

Mi parte fue la búsqueda, la analítica y la depuración automática. Aprendí que un índice de texto en español con pesos permite encontrar «filtraciones» al buscar «filtración» y ordenar por relevancia, y que `explain()` confirma el uso del índice. Los pipelines de agregación me mostraron cómo transformar datos con `$match`, `$dateDiff`, `$group` y `$unwind` sin sacar los datos de la base. Del índice TTL entendí que Mongo borra por sí mismo cuando se cumple el plazo y que el proceso de limpieza corre cada cierto tiempo, por lo que la demostración no es

instantánea. La dificultad fue generar datos de ejemplo coherentes que cumplieran las condiciones del enunciado y verificarlos con un script.

Referencias

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7.ª ed.). https://doi.org/10.1037/0000165-000

Colin McDonnell. (s. f.). *Zod: TypeScript-first schema validation*. https://zod.dev

Meta Platforms. (s. f.). *React*. https://react.dev

MongoDB, Inc. (s. f.-a). *MongoDB manual*. https://www.mongodb.com/docs/manual/

MongoDB, Inc. (s. f.-b). *Schema validation*. https://www.mongodb.com/docs/manual/core/schema-validation/

MongoDB, Inc. (s. f.-c). *TTL indexes*. https://www.mongodb.com/docs/manual/core/index-ttl/

OWASP Foundation. (s. f.). *OWASP Top Ten*. https://owasp.org/www-project-top-ten/

Postman, Inc. (s. f.). *Newman: Command-line integration*. https://learning.postman.com/docs/collections/using-newman-cli/command-line-integration-with-newman/

Supabase. (s. f.). *Storage*. https://supabase.com/docs/guides/storage

Vercel. (s. f.). *Vercel documentation*. https://vercel.com/docs